

Multi-Level Resource-Coherented Graph Learning for Website Fingerprinting Attacks

Bo Gao^{1b}, Graduate Student Member, IEEE, Weiwei Liu^{1b}, Member, IEEE, Guangjie Liu^{1b}, Member, IEEE, Fengyuan Nie^{1b}, Graduate Student Member, IEEE, and Jianan Huang^{1b}, Graduate Student Member, IEEE

Abstract—Deep learning-based website fingerprinting (WF) attacks dominate website traffic classification. In the real world, the main challenges limiting their effectiveness are, on the one hand, the difficulty in countering the effect of content updates on the basis of accurate descriptions of page features in traffic representations. On the other hand, the model's accuracy relies on training numerous samples, requiring constant manual labeling. The key to solving the problem is to find a website traffic representation that can stably and accurately display page features, as well as to perform self-supervised learning that is not reliant on manual labeling. This study introduces the multi-level resource-coherented graph convolutional neural network (MRCGCN), a self-supervised learning-based WF attack. It analyzes website traffic using resources as the basic unit, which are coarser than packets, ensuring the page's unique resource layout while improving the robustness of the representations. Then, we utilized an echelon-ordered graph kernel function to extract the graph topology as the label for website traffic. Finally, a two-channel graph convolutional neural network is designed for constructing a self-supervised learning-based traffic classifier. We evaluated the WF attacks using real data in both closed- and open-world scenarios. The results demonstrate that the proposed WF attack has superior and more comprehensive performance compared to state-of-the-art methods.

Index Terms—Network traffic analysis, website fingerprinting attacks, graph learning, representation learning, self-supervised learning, graph convolutional neural network.

I. INTRODUCTION

THE HTTP protocol, fundamental to the network protocol stack, has swiftly emerged as a preeminent information system on the Internet within a decade [1]. Furthermore, encrypted website traffic has become the most prevalent Internet service in real-world cyberspace [2]. Originally intended for the retrieval of HTML webpages, HTTP is now extensively

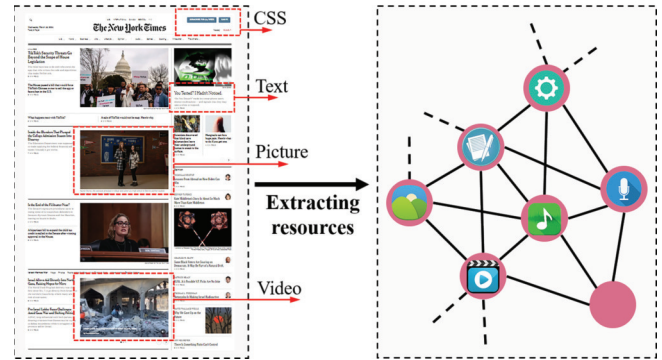


Fig. 1. A relationship graph of the various resources on the website.

employed for transferring various types of hypertext data, encompassing images, audio, and video. Clients successively request each resource file from the web server based on their layout on the page. The server allocates an HTTP channel for each of them, enabling simultaneous delivery to expedite webpage loading. Based on browser parameters, network conditions, and resource sizes, the server dynamically assigns these channels to form differentiated website resource sequences, as shown in Fig. 1.

WF attacks are a crucial task in network traffic analysis, aimed at identifying websites being browsed by clients [3]. In early studies, domain names and IP addresses were used to identify websites [4], [5], [6]. However, with the advent of cloud computing and the growing importance of privacy protection, new technologies such as fast flux service networks (FFSNs) [7], round-robin domain name systems (RRDNS) [8], and content delivery networks (CDN) [9] have gained popularity. They disrupt the steadiness mapping relationship between IPs and websites [10], making character-based WF attacks ineffective, such as deep packet inspection (DPI) [11]. Consequently, researchers have been actively exploring alternative solutions.

Over the past decades, machine learning (ML) algorithms have become a cornerstone of encrypted traffic analysis [12]. Classifiers built using manually crafted features and traditional ML algorithms, such as random forest [13], K-nearest neighbor (KNN) [14], support vector machines (SVM) [15], and the Naive Bayes method [16], have gained significant prominence. However, feature engineering is labor-intensive and can be prohibitively expensive and impractical in open

Received 2 May 2024; revised 6 November 2024; accepted 11 December 2024. Date of publication 18 December 2024; date of current version 7 January 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U2436601, Grant U21B2003, Grant 61602247, and Grant 62072250; and in part by the National Key Research and Development Program of China under Grant 2021QY0700. The associate editor coordinating the review of this article and approving it for publication was Dr. Z. Berkay Celik. (Corresponding author: Weiwei Liu.)

Bo Gao, Weiwei Liu, Fengyuan Nie, and Jianan Huang are with the School of Automation, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: njustgb565@163.com; lwjnjust@njust.edu.cn; niefengyuan@njust.edu.cn; jiananwong@njust.edu.cn).

Guangjie Liu is with the School of Electronics and Information Engineering, Nanjing University of Information Science and Technology, Nanjing 210044, China, and also with the Key Laboratory of Intelligent Support Technology for Complex Environments, Ministry of Education, Nanjing 210044, China (e-mail: gjliu@gmail.com).

Digital Object Identifier 10.1109/TIFS.2024.3520014

1556-6021 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

environments. In contrast, deep learning (DL) algorithms, renowned for their ability to automatically extract and learn features, have demonstrated effectiveness across various fields [17], [18]. High-accuracy classifiers can be achieved through well-designed representation learning, such as byte sequences [19], relational graphs [20], and grayscale images [21].

The quality and scale of the network traffic dataset play a pivotal role in the performance of DL-based WF attacks [22]. In large-scale scenarios, extending neural networks, adding processing units, or expanding datasets to develop classifiers becomes increasingly challenging. First, the sheer number of websites in the real world makes accurate labeling unfeasible. Second, unpredictable network inference and frequent website content updates can cause significant deformations in packet sequences, a phenomenon known as concept drift [23]. DL-based WF methods that rely on raw packet sequences demonstrate weak adaptability to these deformations [24], [25], necessitating frequent model updates and thereby increasing computational costs. Thus, there is an urgent need for a high-precision, automated method for labeling website traffic.

In this paper, we present a self-supervised learning-based WF attack called the multi-level resource-coherented graph convolutional neural network (MRCGCN). It relies on a consensus about websites: the page layout of their resources usually remains stable despite frequent updates to specific content. Compared with the widespread packet feature engineering, we analyzed packet interactions between clients and servers by viewing each resource as a basic unit. The spatio-temporal relationships of website traffic are analyzed at resource-level, flow-level, and host-level, respectively. Six representation sequences are formed to characterize its multi-level structure. Then, an echelon-ordered graph kernel function is utilized to mine page layouts hidden in the traffic. We tagged unlabeled samples with this stable and unique property and extracted auxiliaries such as resource combinations, type combinations, and statistical information for attribute expansion. We cogitated the HTTP and HTTPS protocols's resource transfer mechanisms and network environment fluctuations to create a structurally enrichable extension graph. While emphasizing the intrinsic stability of websites, it reduces external dynamic factors that may affect their accuracy. We design a graph convolutional neural network for the proposed multi-level traffic graph representation by incorporating several graph processing procedures and graph learning mechanisms.

The main contributions could be summarized as follows:

- 1) We proposed a novel multi-level spatio-temporal graph representation for website traffic. It considers all packets originating from the same resource as a whole and consists of resource-level, flow-level, and host-level graphs, which intuitively demonstrate the interactions between clients and servers.
- 2) We developed MRCGCN, a graph convolutional neural network, for the proposed graph representation. First, a graph kernel function is designed for directed graphs with a hierarchical arrangement of nodes, which clusters samples by comparing all nodes on each of the two graphs. The unlearned samples are labeled by calculating the similarity between them and the cluster center.

Finally, the extension graphs formed by the raw graphs and the maximum similarity subgraph are injected into a two-channel network for supervised learning.

- 3) We constructed a dataset containing real HTTP and HTTPS website traffic for evaluating WF attacks in the closed- and open-world scenarios. The results indicate that MRCGCN possesses a well-rounded performance over state-of-the-art methods, including convergence speed, robustness, generalizability, and flexibility.

The remainder of this paper is organized as follows. Section II briefly reviews related work on WF attacks. Section III describes the proposed graph representation for website traffic. The architecture of the MRCGCN is presented in Section IV. In Section V, we benchmarked the proposed WF attack with real-world data. The conclusions are drawn in Section VI.

II. RELATED WORK

WF attacks can generally be divided into three categories based on the learning strategy used: unsupervised learning-based, supervised learning-based, and hybrid learning-based approaches. Each of these strategies extracts traffic features at different levels to form distinct representations, which are then combined with traditional machine learning, deep learning, or graph learning architectures to achieve traffic identification. Table I summarizes the existing WF attacks across four key dimensions: learning strategy, learning architecture, type of features, and traffic representation.

A. Unsupervised Learning-Based WF Attacks

Typical methods include clustering [26], autoencoders [21], anomaly detection [27], hidden Markov models [19], character embedding [28], self-organizing mapping [29], singular value decomposition [30], and principal component analysis [31]. In large-scale scenarios, the huge cost of manually labeling traffic and the importance of security protection have driven researchers to seek unsupervised methods with simplicity of implementation and widespread applicability. In the beginning, each IP address and port were addressed to one website or service [4]. Nevertheless, server clustering services and port mapping techniques breached this uniquely mapping criterion [11]. Protocol fields such as domain names and TLS certificate chains, which indicate the server source, have become the new alternative.

Alqahtani et al. proposed a phishing website detection method called ODAE-WPDC, which injects valid traffic features filtered by an artificial algae algorithm into a deep auto-encoder network training model and achieves 99.28% classification accuracy on an open source phishing website URL dataset [32]. Zhang et al. extracted websites' domain names, certificates, and operator information to construct a heterogeneous graph that visualizes multiple relationships and effectively combats pirated video websites that deployed Domian-Flux technology [33]. Researchers have also been interested in packet feature sequences and their multidimensional statistical attributes.

TABLE I
SUMMARY OF EXISTING WEBSITE FINGERPRINTING ATTACKS

Learning Strategy	Method Name	Learning Architecture	Type of Features	Representation
Unsupervised Learning-based WF Attacks	MRF [19]	Markov Chain	Packet payload	Packet-wise payload byte sequence
	D-PACK [21]	CNN+Autoencoder	Packet payload	Byte grayscale image
	IoT-KEEPER [26]	C-Means	Packet-wise payload byte	Packet-wise feature vectors
	C4.5-WF [27]	Decision Tree	IP/visit time/HTTP message	Protocol field representation
	UE [28]	Huffman Coding	URL length/numeric characters count	URL statistical feature set
	ODAE-WPDC [32]	Auto-Encoder Network	URL length/statistical feature	URL statistical feature set
	HGNN [33]	GNN	IP/third-party service/register	URL relationship graph
Supervised Learning-based WF Attacks	SVM-WF [5]	SVM	URL length/numeric characters/hyphens count	URL statistical feature set
	MAppGraph [10]	GCN	Statistical features of packet length/flow time	Multiple feature vectors
	AppScanner [13]	Random Forest	Packet length	Statistics of packet length sequence
	FineWP [14]	KNN	Packet length	Cumulatively reshaping form of feature sequence
	CUMUL [15]	SVM	Packet length/arrival time	Packet-wise feature vectors
	VNG++ [16]	Navie Bayes	Total trace time/total bandwidth/burst sizes	Flow-wise feature vectors
	Deep Fingerprinting [17]	CNN	Packet direction	Packet-direction sequence
	CNN+LSTM [18]	CNN+LSTM	Packet payload/length	Packet-wise feature vectors
	HMG [20]	Random Forest	URL/redirection/malicious code/HTTP header	HTTP message graph
	Triplet Fingerprinting [22]	CNN+KNN	Packet direction	Statistics of packet direction sequence
	snWF [23]	CNN+ResNet	Packet direction	Packet-direction sequence
	FG-Net [24]	GNN	Packet length/interval time	Flow relationship graph
	Robust Fingerprinting [25]	CNN	Packet direction	Packet-direction sequence
	M2VT-IDS [34]	MLP	Packet payload/length/arrival time	Multi-view representation
	FS-Net [36]	RNN	Packet length	Multiple feature vectors
	3-DNNs [37]	CNN+KNN	Packet length/direction	Packet-wise sequences
	PERT [38]	Transformer	Packet payload/length	Dynamic word embedding
	GraphDApp [39]	GNN	Packet length/direction	Traffic interaction graph
	RK-HSTGCN [40]	GCN	HTTP command/extension/packet arrival order	Resource knowledge-driven representation graph
	SML-WF [42]	Bayes Net, Random Forest	Packet length	Statistics of packet length sequence
Hybrid Learning-based WF Attacks	STC-WF [51]	GNN	Packet length	Inter-flow spatio-temporal correlation graph
	GAP-WF [52]	GAP	Packet length/direction/interval time	Packet-wise feature vectors
	FM-CWFA [35]	CNN	Packet direction/arrival time	Packet-wise feature vectors
	DCGAN [43]	GAN	Packet length/arrival time sequence	Packet-wise feature vectors
	FlowPrint [44]	Random Forest	TLS certificate/IP/packet length/arrival time/direction	Multiple feature vectors
	FS-GAN [45]	GAN	Packet payload	Packet payload sequence
	EDRL [46]	CNN	Packet length	Packet-wise feature vectors
	App-Net [47]	CNN+LSTM	Packet payload/length	Packet-wise feature vectors
	SoK [48]	SVM, CNN	Packet arrival time	Intervals of packets
	MRCGCN	GCN	Webpage layout/Resource-related spatio-temporal features	Resource-coherented representation graph

B. Supervised Learning-Based WF Attacks

Notable supervised learning methods include random forest [13], SVM [15], MLP [34], CNN [35], RNN [36], LSTM [37], transformer [38], and GNN [39], [40]. Constructing classifiers by combining MLs with traffic representations has emerged as a generic paradigm for resolving website recognition tasks [3]. Early WF attacks inductively summarized the demarcation conditions for classifying website traffic types by learning artificially designed traffic features from massive labeled datasets [41].

Taylor et al. proposed the AppScanner to identify Android apps in real time by inputting 54 statistical features extracted from the uplink, downlink, and bidirectional packet sequences into a random forest classifier [13]. Nevertheless, this pattern is highly dependent on expert knowledge, and the models have inadequate generalization capabilities, which restricts their application in large-scale and automated scenarios [42].

In recent years, DLs have gained popularity in network traffic analysis due to their ability to automatically extract features, especially as represented by end-to-end deep network frameworks [17]. Rimmer et al. systematically analyzed

various DLs applied to WF, including feedforward, convolutional, and recurrent neural networks, and designed three common models, SDAE, CNN, and LSTM [37]. The results show that end-to-end DLs are more advantageous in automatically extracting traffic features than manual feature extraction, but they require bulky labeled samples for training models.

C. Hybrid Learning-Based WF Attacks

The network environment and experimental settings in traffic analysis tasks are inherently complicated, making it challenging for a single model to sufficiently satisfy the diverse requirements. Consequently, the researchers proposed various hybrid learning schemes. Expecting to conquer the practical difficulties by combining the advantages of multiple algorithms, such as semi-supervised learning [43], [44], self-supervised learning [45], transfer learning [35], reinforcement learning [46], multimodal learning [47], and GANs [48]. DCGAN is a semi-supervised learning method based on deep convolutional generation adversarial networks, which uses a dataset with generator-generated samples and unlabeled data [43]. FS-GAN enables the generated massive samples to have

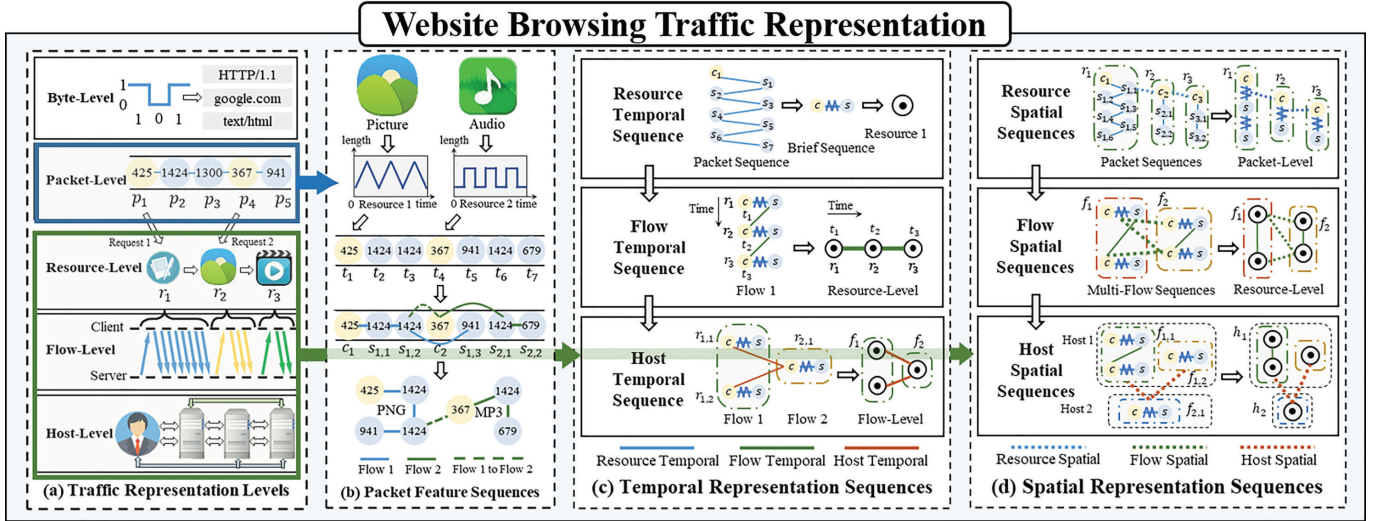


Fig. 2. Multi-level spatio-temporal representations of the website browsing traffic. (a) Traffic Representation Levels; (b) Packet Feature Sequences; (c) Temporal Representation Sequences; (d) Spatial Representation Sequences.

a distribution similar to the real data by carefully designing the generative adversarial network [45].

In summary, most unsupervised learning-based WF attacks rely on character-level features, which are often unreliable and unstable in practice. Encryption of content, domain name obfuscation, and dynamic IP addresses can obscure or alter many plaintext features. Traditional ML algorithms, which depend on manual feature engineering, are costly and difficult to scale in real-world scenarios. While supervised DL algorithms offer high accuracy and ease of use, they require continuous training on large datasets, making them challenging to implement due to the need for constant and accurate labeling in open-world environments. To address this, it is essential to identify a learning strategy that balances classification accuracy with temporal robustness. Hybrid learning approaches, such as self-supervised learning, semi-supervised learning, transfer learning, and reinforcement learning, have been identified as promising solutions for automated, large-scale scenarios. Graph-based methods, which convert non-Euclidean data with complex logical relationships into comprehensible graphical structures, are particularly appealing. By mapping traffic data to graph representations, the key structural information is highlighted while fine-grained traffic details are abstracted, thereby enhancing model robustness and generalizability. The primary challenge now lies in accurately mapping the spatio-temporal relationships within traffic feature sequences to graph representations. Therefore, it is critical to explore efficient fusion methods that incorporate prior knowledge into graph representations and to apply suitable graph learning algorithms to refine core traffic features.

III. WEBSITE BROWSING TRAFFIC REPRESENTATION

In this section, we introduce a multi-level spatio-temporal graph representation for website traffic. First, we consider resources as basic units and construct six temporal and spatial feature sequences to describe the interaction between clients and servers at the resource-level, flow-level, and host-level,

TABLE II
LIST OF NOTATIONS

Notation	Meaning
W	One website browsing behavior.
$W^{[T]}, W^{[S]}$	Temporal and spatial representations of W .
P, p_i	A packet sequence and the i -th packet in it.
R, r_i	A resource sequence and the i -th resource in it.
F, f_i	A flow sequence and the i -th flow in it.
H, h_i	A host sequence and the i -th server IP in it.
$\mathcal{F}, \eta$	Packet features and feature sets.
l, t, d	Length, arrival time, and direction of a packet.
G	The graph representation of W .
V, E	The node set and edge set on G .
v, e	A node and an edge on G .
τ_p	The arrival time interval between two packets.
σ, ι	The graph depth and resource out-degree of v .
ϕ	Nodes' aggregation function.
ψ	Edges' update function.
κ	The echelon-ordered graph kernel function.
g_κ	A similar subgraph extracted from two graphs.
ϱ	Scan for nodes with the same features.
Δ	Similarity of graphs.

respectively. Second, the graph is constructed with resources as nodes and multi-level relationships among them as edges. At this point, we have transformed the website traffic classification problem into a graph recognition problem.

A. Extraction of Website Traffic Representation Elements

Commonly in traffic packet engineering, one website browsing behavior W could be represented by a packet sequence, denoted as $W = P = (p_1, p_2, \dots, p_n)$, where p_i denotes the i -th packet, as shown in Fig. 2(b). Generic packet features η_{p_i} include length l_i , arrival time t_i , and transmission direction d_i , denoted as $\eta_{p_i} = \{l_i, t_i, d_i\}$. The notations used in this paper are shown in Table II.

Despite this basic website traffic representation's convenience, the generated sequences lack notable robustness and are susceptible to deformation from the network environment, transmission lines, and communication devices.

To create a valid and robust traffic representation, we integrated the structural characteristics of generic websites with their actual browsing traffic. According to imec-DistriNet [49], the daily fluctuation for the top 100,000 websites on statistical platforms like Alexa [50] is less than 1%, making it feasible to use a one-day list for long-term observation. We first organized the resources based on the logical structure of the webpage code obtained via a web crawler. Then, a browser driver was used to simulate client browsing behavior, capturing long-term traffic for each website. Finally, we compared the order of resource transmission with its logical structure.

During validation, we used Wireshark's decryption function to verify the transmission order of encrypted HTTPS traffic. Our analysis of numerous generic websites confirmed a key observation: the resource transmission order consistently follows the page layout, representing a unique and stable intrinsic feature for each website. Notably, this order exhibits dynamic stabilization, meaning that while multiple resources in the same state are transmitted within a given time period, the specific order may change dynamically [51]. Even with HTTPS encryption, the resource ordering remains consistent with plaintext HTTP/1.1, where resources are transmitted sequentially. In this case, all packets of a resource must be fully transmitted before the next resource is sent. Incorporating this essential characteristic into the traffic feature sequence greatly enhances the robustness of the traffic representation.

Figure 2(a) shows five granularities of website traffic from fine to coarse: byte, packet, resource, flow, and host. We constructed six representation sequences with resources as the basic unit, as shown in Figs. 2(c) and 2(d). The temporal dimension focuses on ordering, including resource-, flow-, and host-level temporal sequences, denoted as $W^{[T]} = \{R^{[T]}, F^{[T]}, H^{[T]}\}$. The spatial dimension accounts for transmission time overlap, denoted as resource-, flow-, and host-level spatial sequences $W^{[S]} = \{R^{[S]}, F^{[S]}, H^{[S]}\}$.

1) *Resource-Level Temporal Sequence $R^{[T]}$* : A website resource r generally consists of a request packet c and several response packets s , denoted as $r = (p_1, p_2, \dots, p_n)$. Page layout determines the initial request packet's arrival time, whereas others are resource-related. Therefore, we labeled each resource with that time and arranged them in a sequence, denoted as $R^{[T]} = (r_1, r_2, \dots, r_m)$, where r_j denotes the j -th resource, and its arrival time is denoted as $t(r_j) = t(p_{j,1})$.

2) *Resource-Level Spatial Sequence $R^{[S]}$* : HTTP/1.1 websites utilize the pipelined persistent connection strategy, which transfers resources in parallel, to accelerate page loading. Despite this resulting in variable combinations of resources for each browse, resources in the same section of the page are adjacent in the traffic and have overlapping transfer times, owing to the website code and page layouts.

To represent this worthy spatial feature, we proposed resource-level spatial sequences, denoted as $R^{[S]} = \{(r_1, p_{1,i}), (r_2, p_{2,i}), \dots, (r_j, p_{j,i}), (r_m, p_{m,i})\}$, where $p_{m,1}$ denotes the first packet in resource r_m , and its

arrival time is located in packets p_{ji} and p_{ji+1} in resource r_j . Note that this only records the first intersection.

3) *Flow-Level Temporal Sequence $F^{[T]}$* : Flow affiliation is also a meaningful spatial feature, besides the resource affiliation. Flow representation sequence is denoted as $F^{[T]} = (f_1, f_2, \dots, f_k)$, where $f_k = (r_{k,1}, r_{k,2}, \dots, r_{k,m})$ denotes the k -th flow, and $r_{k,1}$ denotes the first resource. HTTP flows transmit only one resource at a time; therefore, the packets of neighboring resources are sequenced tail-to-head.

4) *Flow-Level Spatial Sequence $F^{[S]}$* : It represents the interleaving relationship of resources at flow-level, denoted as $F^{[S]} = \{(f_1, r_{1,j}), (f_2, r_{2,j}), \dots, (f_k, r_{k,j}), (f_g, r_{g,m})\}$, where the start time of resource $r_{g,m}$ is located in resources $r_{k,j}$ and $r_{k,j+1}$. Notably, if the start times of resources $r_{g,m}$ and $r_{g,m+1}$ are located within the duration of resource $r_{k,j}$, then they are all related to that resource.

5) *Host-Level Temporal Sequence $H^{[T]}$* : It is described as $H^{[T]} = (h_1, \dots, h_m)$, where $h_i = (f_{i,1}, \dots, f_{i,k}) = (r_{i,1}, \dots, r_{i,n})$ denotes the i -th host, and $f_{i,1}$ denotes its first flow. Remarkably, when the count of resources exceeds the maximum threshold of HTTP channels, the leftover resources are suspended. Thus, the host representation sequence with resource affiliation is more stable than flow affiliation.

6) *Host-Level Spatial Sequence $H^{[S]}$* : It indicates the interleaving relationship of resources at host-level, denoted as $H^{[S]} = \{(h_1, r_{1,j}), (h_2, r_{2,j}), \dots, (h_k, r_{k,j}), (h_g, r_{g,m})\}$, which facilitates revealing hidden arcanum on different hosts.

B. Construction of Graph Representations

Graph: $G = (V, E)$ denotes a graph, where V represents a set of nodes and each node v_i corresponds to a packet p_i . $e_{i,j} \in E$ denotes a set of edges, where the edge $e_{i,j}$ connects the nodes v_i and v_j , indicating the relationship between them. The packet-level graph $G^{[P]}$ is an initial graph composed of packets, as shown in Fig. 3(a).

Node: The initial features of a node are derived from the actual traffic, the three generic properties of a packet, denoted as $\eta_{v_i} = \eta_{p_i} = \{l_i, t_i, d_i\}$, which reflect the fine-grained information interaction between the client and the server at the packet level. In reality, the packet feature sequence is fragile and easily affected by the network environment. The graphs $G^{[P]}$ generated each time vary widely, making classification unfeasible. To resolve this, we utilized resource affiliation instead of packet serial numbers and expanded node features to $\eta'_{v_i} = \eta_{v_i} \oplus \{r, f, h\}$, where r, f , and h denote the resource, flow, and host serial numbers, respectively, and the splice symbol $\oplus$ indicates injecting new features into the raw feature set η_{v_i} . These three features are spatial properties derived from full-cycle observations of the real website traffic, and their injection facilitates the revelation of intricate high-dimensional logical relationships among resources.

Edge: The graph has two primary types of edges: temporal $e^{[t]}$ and spatial $e^{[s]}$, which could be further divided into resource, flow, and host-levels, denoted as $E = \{e^{[t]}, e^{[s]}\} = \{[e^{[r,t]}, e^{[f,t]}, e^{[h,t]}], [e^{[r,s]}, e^{[f,s]}, e^{[h,s]}]\}$.

1) *Resource Temporal Edge $e^{[r,t]}$* : It shows the temporal relationship of packets within each resource. For example, if packets p_i and p_j belong to the same resource r_m and their

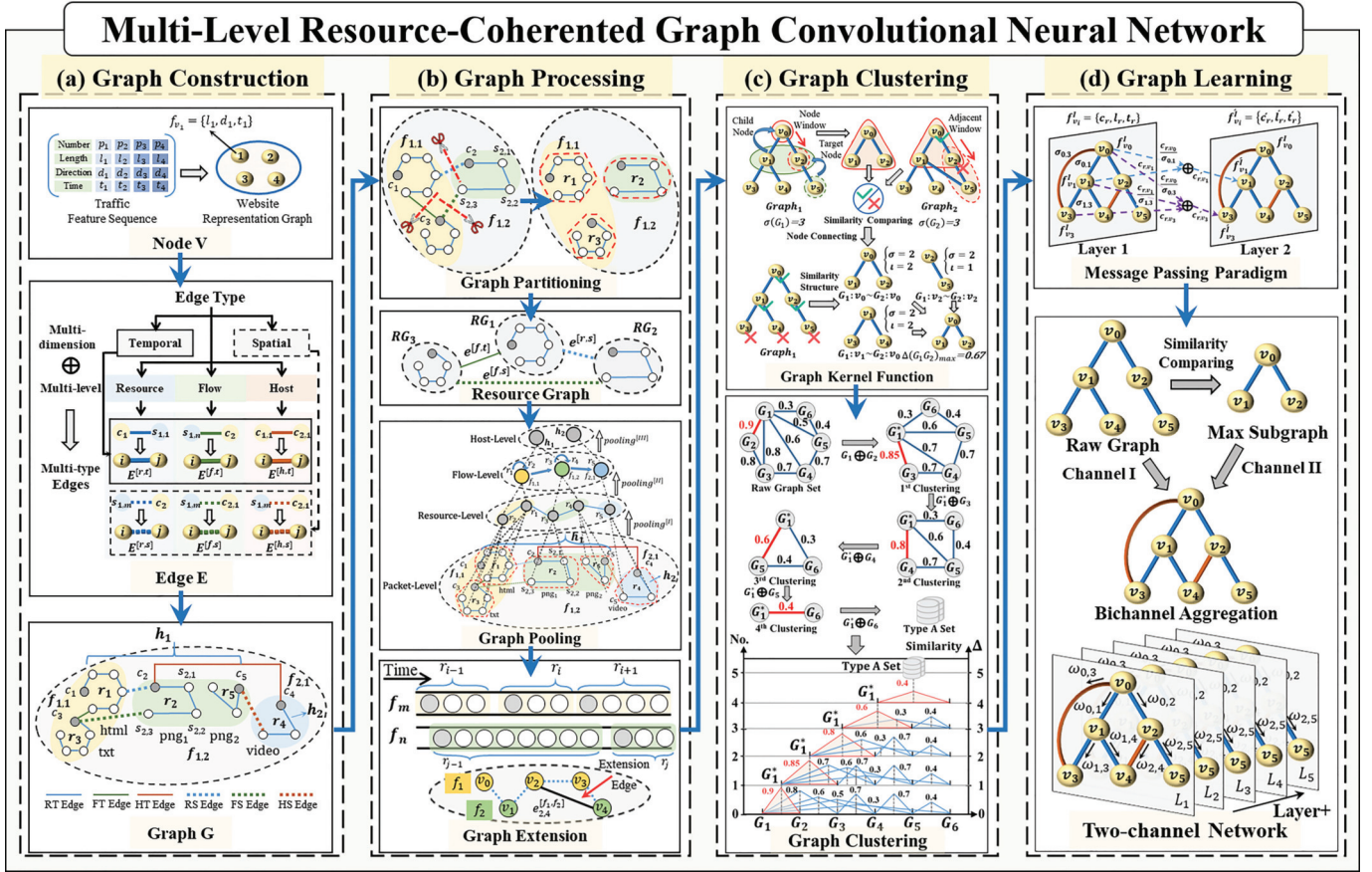


Fig. 3. Architecture of the proposed multi-level resource-coherented graph convolutional neural network (MRCGCN). (a) Graph Construction; (b) Graph Processing; (c) Graph Clustering; (d) Graph Learning.

arrival times are neighboring, they would be renumbered as packets $p_{m,i}$, and $p_{m,i+1}$. The edge between them is denoted as $e_{i,i+1}^{[r,m,t]}$, with feature $\eta_{e^{[r,m,t]}} = \tau_p$, where τ_p denotes the arrival time interval between two packets.

2) *Flow Temporal Edge* $e_{i,i+1}^{[f,t]}$: Depending on the start time, resources within the same flow are connected to form flow temporal edges, which are denoted as $e_{m,m+1}^{[f,t]}$, with feature $\eta_{e^{[f,t]}} = \tau_r$, where τ_r denotes the interval between the end time of resource r_m and the start time of resource r_{m+1} .

3) *Host Temporal Edge* $e_{i,i+1}^{[h,t]}$: Considering stability, we constructed host temporal edges at resource-level, which are denoted as $e_{m,m+1}^{[h,t]}$, with feature $\eta_{e^{[h,t]}} = \tau_h$, where m denotes the starting node of m -th resource in g -th host, and τ_h denotes the time interval between resources r_m and r_{m+1} .

4) *Resource Spatial Edge* $e_{i,i+1}^{[r,s]}$: If neighboring nodes v_i and v_{i+1} belong to different resources, and v_{i+1} is the first node of a new resource r_{m+1} , redefine the edge $e_{i,i+1}$ between them as a resource spatial edge, denoted as $e_{i,i+1}^{[r,s]}$. Its feature locates transmission time overlap between two resources, denoted as $\eta_{e^{[r,s]}} = \delta_p$, where $\delta_p = i$ indicates that resource r_{m+1} is transmitted after i -th packet of resource r_m .

5) *Flow Spatial Edge* $e_{i,i+1}^{[f,s]}$: If resources r_m and r_{m+1} belong to different flows, connect them to derive a flow spatial edge, denoted as $e_{m,m+1}^{[f,s]}$. The connected nodes are the i -th node $p_{m,i}$ in resource r_m and the first node $p_{m+1,1}$ in resource r_{m+1} with $t_{p_{m,i}} < t_{p_{m+1,1}} < t_{p_{m,i+1}}$. Its feature is denoted as $\eta_{e^{[f,s]}} = \delta_r$.

Notice that $\delta_r = i$ is the same as resource spatial edges. In subsequent graphs, $\delta_r = m$ denotes that resource $r_{g,n}$ in flow f_g is transmitted after m -th resource $r_{k,m}$ in flow f_k for highlighting the interleaving at flow-level.

6) *Host Spatial Edge* $e_{i,i+1}^{[h,s]}$: It is analogous to the flow spatial edges, connecting neighboring resources r_m and r_{m+1} that belong to different hosts, denoted as $e_{m,m+1}^{[h,s]}$. Its feature is denoted as $\eta_{e^{[h,s]}} = \delta_r$, whose meaning is the same in both graphs $G^{[P]}$ and $G^{[R]}$. In graph $G^{[F]}$, $\delta_r = g$ indicates that a resource is transmitted by host h_l after a resource in host h_g .

The six edges in the graph $G^{[P]}$ are pointed from the node transmitted first to the latter. Analyzing numerous websites indicated that cohort nodes tend to be sent together. Introduced extra node features include graph depth σ and resource out-degree ι , denoted as $\eta''_{v_i} = \eta'_{v_i} \oplus \{\sigma, \iota\}$. Resource out-degree refers to the count of edges connected to different resource nodes. The depth increment direction is the same as the edge pointing. All nodes belong to the same resource share graph depth σ and resource out-degree ι . These two features can directly reflect the graph's structure, the connection mode of nodes and edges, and will serve as the core characteristics for executing graph clustering.

IV. GRAPH CONVOLUTIONAL NEURAL NETWORK

We propose the graph convolutional neural network (MRCGCN) in this section. We first optimized the graph $G^{[P]}$

created in Section III to make it appropriate for automated and large-scale applications. Second, a graph clustering algorithm is proposed that assigns pseudo-labels to partial website traffic based on graph structural similarities. The website traffic is labeled based on the similarity of the samples to the clustering center, which could reduce the computational expense. To achieve accurate categorization, a two-channel graph neural network is also implemented, which enhances the generalization of the graph by aggregating the separately optimized raw graph and maximum similarity subgraph.

A. Graph Processing

With the Internet's rapid growth, image pixels and video clarity constantly increase, and more packets are required to transmit them, rendering the graphs' size uncontrollable [52]. This subsection describes graph optimization processing, including graph partitioning, constructing resource graphs, graph pooling, and graph extension, as shown in Fig. 3(b).

Graph Partitioning: It divides a huge graph into subgraphs that could be performed in parallel on one system. The initial graphs $G^{[P]}$ usually contain massive nodes and spatio-temporal edges, which require judicious selection of appropriate data reading methods and graph partitioning algorithms.

Streaming Partitioning Algorithm: According to resource affiliation, we divide the graph $G^{[P]}$ into subgraphs and associate a separate processing unit for each subgraph. The stream segmentation algorithm supports reading data in batches and gradually adding all nodes and edges to new subgraphs on the basis of resource serial numbers.

Node Partitioning Algorithm: Node and edge partitioning are universal graph segmentation strategies [53]. The former trims edges to preserve subgraph nodes. The latter copies extra nodes into each subgraph to preserve edges. In the graph $G^{[P]}$, except for resource temporal edges, the others were not involved in the packet-level node feature computation and could be converted into subgraph features. We select the node segmentation approach since it ensures data integrity and maximizes processing time by parallelizing on several computers.

Resource Graph: Graph partitioning creates a resource graph with all nodes from the same resource [54], denoted as $RG = (V', E', F')$, where V' denotes a set of all nodes v'_i from the same resource, E' denotes the set of edges, and edges $e'_{i,j}$ correspond to the resource temporal edges $e^{[r,t]}$ in the graph $G^{[P]}$. The graph features are denoted as $\mathcal{F}' = \cap \eta'_{v'_i} = \{r, f, h, \sigma, \iota\}$, where $\cap$ denotes the common feature of all packet nodes.

The graph $G^{[P]}$ could be represented as a sequence of resource graphs, denoted as $G^{[P]} = (RG_1, RG_2, \dots, RG_n)$, where RG_i denotes the i -th resource graph. Its relationship sequence is derived from the five types of edges in the graph $G^{[P]}$, denoted as $S(G^{[P]}) = [(RG_1, RG_2, s_1), \dots, (RG_{n-1}, RG_n, s_k)]$, with $k > n - 1$, where s_j denotes the attribute relationship between the resource graphs RG_m and RG_n .

Graph Pooling: We presented an affiliation-driven hierarchical graph-based pooling algorithm that aggregates the same

affiliation nodes to reduce graph size [55]. It has four levels: packet, resource, flow, and host, defined as Eq. (1).

$$G^{[H]} = \text{pool}^{[III]}(G^{[F]}) = \text{pool}^{[II]}(G^{[R]}) = \text{pool}^{[I]}(G^{[P]}) \quad (1)$$

where graphs $G^{[H]}$, $G^{[F]}$, $G^{[R]}$, and $G^{[P]}$ denote the host-, flow-, resource-, and packet-level subgraphs, respectively.

Eq. (2) denotes the nodes aggregation function ϕ .

$$v_j^{[l+1]} = \phi(A : v_i^{[l]}), \quad \eta_{v_j^{[l+1]}}(A) = j \quad (2)$$

where ϕ aggregates all nodes $v_i^{[l]}$ on layer l with common affiliation A to the j -th node $v_j^{[l+1]}$ on layer $l + 1$.

Edges update function ψ extracts all edges on layer l combined into edges $E^{[l+1]}$, and is defined as Eq. (3).

$$E^{[l+1]} = \psi(m : e^{[l]}), \quad m > 1 \quad (3)$$

The pooling operations between layers are described below:

1) **First-Level Pooling $\text{Pool}^{[I]}$:** Using a resource node to replace all packet nodes within it, construct a resource-level subgraph, denoted as $G^{[R]} = (V_R, E_R)$, where V_R denotes a set of resource nodes $v_i^{[r]}$ and E_R denotes a set of edges $e_{i,j}^{[r]}$. Eqs. (4) and (5) define their pooling functions.

$$v_j^{[r]} = \phi(R : v_i^{[p]}), \quad \eta_{v_j^{[r]}}(r) = \cup \eta_{v_i^{[p]}}(r) \quad (4)$$

$$e^{[r]} = \psi(e^{[p]} : ([f, t], [h, t], [r, s], [f, s], [h, s])) \quad (5)$$

where $v_i^{[p]}$ and $e^{[p]}$ denote nodes and edges in the graph $G^{[P]}$, respectively, and $\cup$ denotes aggregating the features of all packet-level nodes within a resource.

The resource node feature is denoted as $\eta_{v_i^{[r]}} = \cup \eta''_{v_i^{[p]}} = (c_r, l_r, t_r, r, f_r, h_r, \sigma_r, \iota_r)$, where c_r denotes the account of packets, l_r denotes the accumulated length, and t_r denotes its starting time. r , f_r , and h_r denote its resource, flow, and host serial number, respectively. σ_r denotes resource depth, and ι_r denotes resource out-degree.

2) **Second-Level Pooling $\text{Pool}^{[II]}$:** The flow-level subgraph $G^{[F]}$ regards all resource nodes as one flow node, denoted as $G^{[F]} = (V_F, E_F)$, where V_F denotes a set of flow nodes $v_i^{[f]}$, and E_F denotes a set of edges $e_{i,j}^{[f]}$. Eqs. (6) and (7) define node and edge pooling functions.

$$v_j^{[f]} = \phi(F : v_i^{[r]}), \quad \eta_{v_j^{[f]}}(f) = \cup \eta_{v_i^{[r]}}(f) \quad (6)$$

$$e^{[f]} = \psi(e^{[r]} : ([f, t], [h, t], [f, s], [h, s])) \quad (7)$$

where $v_j^{[f]}$ denotes the j -th node in the graph $G^{[F]}$, and $\eta_{v_i^{[f]}} = \cup \eta''_{v_i^{[r]}}$ denotes its feature.

3) **Third-Level Pooling $\text{Pool}^{[III]}$:** All flow nodes under the same host are aggregated into a host node to construct a host-level subgraph $G^{[H]}$, denoted as $G^{[H]} = (V_H, E_H)$, where V_H denotes a set of host nodes $v_i^{[h]}$, and E_H denotes its edges $e_{i,j}^{[h]}$. Eqs. (8) and (9) define their pooling functions.

$$v_j^{[h]} = \phi(H : v_i^{[f]}), \quad \eta_{v_j^{[h]}}(h) = \cup \eta_{v_i^{[f]}}(h) \quad (8)$$

$$e^{[h]} = \psi(e^{[f]} : ([h, t], [h, s])) \quad (9)$$

where $v_j^{[h]}$ denotes the j -th node in the graph $G^{[H]}$, and $\eta_{v_i^{[h]}} = \cup \eta''_{v_i^{[f]}}$ denotes its feature.

Using affiliation-driven hierarchical graph-based pooling, a multi-level resource-coherented graph is created. All pooled

graphs preserve the raw information at equal and higher levels, substituting lower levels with statistical results. This pooling strategy could mine crucial website traffic features at several scales and adapt to various website types.

Graph Extension: The realistic network environment affects resource combinations, making graph clustering difficult. We proposed graph extensions based on HTTP protocol transmission mechanisms and webpage layouts. Additional flow-level edges are added into the raw graph to increase correlation, thus making the graphs of the same website more similar [56].

The extension graph is defined as $G^{[R]} = (V_R, \hat{E}_R)$, and the edge set $\hat{E}_R$ includes several new flow-level edges $e_{i,j}^{[f_m, f_n]}$. If the starting time of five nodes is represented as $t_{i-1} < t_{j-1} < t_i < t_{i+1} < t_j$, where nodes v_{i-1} , v_i , and v_{i+1} all belong to flow f_m , nodes v_{j-1} and v_j belong to flow f_n , and all belong to the same host. Then nodes v_i and v_j are connected using an extension edge, which is defined as $e_{i,j}^{[f_m, f_n]}$. Extension edges show the order of resource transfers within different flows on the same host, which also belongs to temporal edges, and its feature could be denoted as $\eta_{e_{i,j}^{[f_m, f_n]}} = \tau_r$, where τ_r denotes the starting time interval of resource r_i and r_j .

As extension edges are established based on website knowledge, they are less relevant to traffic shape than the others. Hence, all edges in the raw graph are defined as I-type, and extension edges are defined as II-type, which serve as substitutes for I-type edges in graph clustering.

B. Graph Clustering

This subsection proposes a kernel function for echelon-ordered graphs, which scans the graphs two-by-two for the maximum similarity structure. Subsequently, a coalescent hierarchical clustering algorithm is employed to categorize unlabeled traffic samples, as shown in Fig. 3(c).

The clustering object in this subsection is the resource-level layer of the multi-level resource-cohered graph. Compared to the flow-level and host-level layers, it demonstrates a more intricate graph structure and reveals detailed spatio-temporal variations in website traffic. Furthermore, its structure is more resilient than packet-level layers, and the reduced graph sizes facilitate the execution of node-level graph computations.

Graph Kernel Function: The echelon-ordered graph kernel function κ is geared towards directed graphs where nodes exist in a hierarchy. It compares each node on two graphs to find the maximum similarity subgraph and computes similarities.

In a graph space $\hat{G}$, the graph representation for website traffic W_a is denoted as $G_a = (V_a, E_a)$. The set of similar subgraphs between two graphs is defined as Eq. (10).

$$G_K(G_a, G_b) = \sum_{v_i \in V_a} \sum_{v_j \in V_b} g_K(v_i, v_j) \quad (10)$$

where $g_K(v_i, v_j)$ denotes the extracted similar subgraphs starting with nodes v_i and v_j , and $G_K(G_a, G_b)$ denotes an ensemble of all similar subgraphs in two graphs G_a and G_b .

Similar subgraphs are built by linking isomorphic nodes on adjacent hierarchies of each graph, defined as Eq. (11).

$$g_K(v_i, v_j) = \oplus\{\kappa(v_i^m, v_j^m)\}, \quad m \in [1, n] \quad (11)$$

where κ scans out nodes with the same resource out-degree in the set of m -th hierarchies neighborhood nodes of nodes v_i^m and v_j^m . The splice symbol $\oplus$ connects isomorphic nodes at all hierarchies to form a similar subgraph $g_K(v_i, v_j)$.

The graph kernel function κ is defined as Eq. (12).

$$\kappa(v_i^m, v_j^m) = \varrho((\eta^I : \iota(v_i^m), \iota(v_j^m)), \kappa(v_i^{m+1}, v_j^{m+1})) \quad (12)$$

where the scanning function ϱ picks nodes with the same I-type edges. If the I-type edge of node v_i^m is less than that of node v_j^m , its II-type edges are used to substitute.

Graph Similarity. Both raw graphs and the maximum similarity subgraphs are trees. Higher overlap between them indicates a similar website. The similarity formula is defined as Eq. (13).

$$\Delta(G_a, G_b) = \frac{\sigma_{\max}(G_K)}{\sigma(G_a) + \sigma(G_b) - \sigma_{\max}(G_K)} \quad (13)$$

where $\Delta(G_a, G_b)$ denotes the similarity between graphs G_a and G_b , $\sigma(G_a)$ denotes the depth of graph G_a , and $\sigma_{\max}(G_K)$ denotes the depth of the maximum similarity subgraph G_K .

We utilized the similarity-weighted average $\bar{\Delta}$ of resource-level, flow-level, and host-level subgraphs as the final similarity between graph G_a and G_b , defined as Eq. (14) [57].

$$\bar{\Delta} = \alpha\Delta_R + \beta\Delta_F + (1 - \alpha - \beta)\Delta_H \quad (14)$$

where Δ_R , Δ_F , and Δ_H denote resource-, flow-, and host-level similarities, respectively, and α , β denote the coefficients.

Graph Clustering. The coalescent hierarchical clustering algorithm merges the maximum similarity samples from the bottom layer to form a new upper layer until the aggregation termination condition is reached.

Algorithm 1 shows the graph's hierarchical clustering procedure. First, each graph is considered a category, and the graph kernel function calculates the similarity between each two graphs. Secondly, these graphs are replaced with the maximum similarity subgraph, and their serial numbers are merged into the new graph. Recalculating the new graph's similarities with the others is the final step. Repeat steps two and three until the aggregation termination condition is reached.

C. Graph Learning

In this subsection, a two-channel heterogeneous graph convolutional neural network for WF is proposed. For training and classifier generation, the raw graph and its subgraph would be inputted into different network channels simultaneously.

Message Passing Paradigm: We designed a directed aggregation function for node features based on the hierarchy deviation to completely reflect the relationship between nodes. Node features on the graph can be grouped into three types: spatio-temporal features, traffic affiliation, and node status.

For example, the node features of the graph $G^{[R]}$ are represented as $\eta_{v_i^{[r]}} = (c_r, l_r, t_r, r, f_r, h_r, \sigma_r, \iota_r)$. The packet count c_r , accumulated length l_r , and start time t_r belong to I-type node features that indicate the traffic shape. Resource, flow, and host serial numbers (r , f_r , h_r) are II-type node features that identify packet affiliation at varying granularities. III-type node features, graph depth σ , and resource out-degree ι , are

Algorithm 1 Graph Clustering Algorithm**Input:**

The graph space $\hat{G}$;
 Clustering termination conditions: minimum number of clusters N_{min} , or the global minimum similarity Δ_{min} allowed to be categorized as a same category.

Output:

The number of clusters N ,
 The serial numbers of all graphs within each cluster $S_i = \{G_1, \dots, G_n\}$,
 The minimum similarity of each cluster Δ_i ,
 The maximum similarity subgraphs within each cluster $G_{\kappa|\sigma_{max}}$.

- 1: **Initialize:** $S_i = \{\}$, $\Delta_{N_i} = 0$
- 2: **while not converged do**
- 3: Step 1) The similarity subgraph between every two graphs G_a, G_b is extracted by Eqs. 10–12;
- 4: Step 2) Calculate the similarity between each two graphs $\Delta(G_a, G_b)$, according to Eq. 13;
- 5: Step 3) Aggregate the two graphs that hold the highest similarity Δ_{max} and replace them with the maximum similarity subgraphs $G_{\kappa|\sigma_{max}}$, whose serial numbers are merged into the new graph $S_i = \{a, b\}$;
- 6: Check the aggregation termination condition:
 $N = N_{min}$, or $\forall \Delta_i = \Delta_{min}$.
- 7: **end while**
- 8: **return** $N, S_i, \Delta_i, G_{\kappa|\sigma_{max}}$

utilized for graph matching to uncover page resource layout by nodes in any two graphs.

The lesser hierarchy deviation between nodes, the stronger their association. The directed aggregation function weights the node I-type features based on the resource out-degree deviation and takes the result as the I-type node features of the nodes in the new graph, which is defined as Eq. (15).

$$\hat{\eta}_{v_j}^I = \eta_{v_j}^I + \sum_{i \in N(j)} \frac{\eta_{v_i}^I}{\sigma_{v_j} - \sigma_{v_i}} \quad (15)$$

where $\eta_{v_j}^I$ denotes the I-type features of node v_i , $N(j)$ denotes the set of neighboring nodes with node v_j , and $\hat{\eta}_{v_j}^I$ denotes the new I-type node feature of the aggregation.

Two-channel Network: The two-channel model first inputs the raw graph and the maximum similarity subgraph, each with a separate channel. Whereafter, the two input graphs are optimized independently, including graph partitioning, graph pooling, and graph extension. The raw graph is expanded using real traffic packet spatio-temporal distribution. The maximum similarity subgraph extends itself according to the category it belongs to. We reverse-order the graphs and clustering centers of the processing in graph clustering and set the subcenter A_k , where A_k denotes the similar structure of the penultimate clustering output. In practice, we select the subcenter that is injected into the maximum similar subgraph to topologize the graph structure in accordance with the dataset size.

We aggregated the graphs in the two-channel with structure and feature fusion. First, the maximum similarity subgraphs

might generate new nodes and edges during graph extension. Edges are joined first in raw graphs with nodes but no edges. If no nodes are related, the hierarchy deviation algorithm advises alternatives with similar structure and the lowest hierarchy deviation. Then, the node and edge features of the raw graph and the maximum similarity subgraph are weighted based on the serial number to achieve feature fusion.

Furthermore, we designed an HTTP protocol coloring module to improve MRCGCN's applicability. When the website traffic is HTTP protocol, the node feature is expanded to $\eta_{v_i}''' = \eta_{v_i}'' \oplus \{\gamma, \epsilon\}$, where γ denotes resource format and ϵ represents its type. This processing could guide the graph neural network to focus on special resource type nodes, such as videos or audios.

Fully-Connected Layer: It is connected after the graph convolution layer with a linear transformation function. We employed the *SoftMax* function Eq. (16) to classify results.

$$\hat{y}_{ic} = \text{SoftMax}(\text{MRCGCN}(x_i)) \quad (16)$$

where x_i indicates the i -th graph $G^{[R]}$, and $\hat{y}_{ic}$ indicates its predicted label as the class c .

The cross-entropy loss function is defined as Eq. (17), and *ReLU* is chosen as the activation function.

$$\text{Loss} = -\frac{1}{|X|} \sum_{i=1}^{|X|} \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic}) \quad (17)$$

where y_{ic} denotes the label c of x_i , C denotes the website type number, and $|X|$ denotes the count of training samples.

V. EXPERIMENTAL RESULTS AND ANALYSIS

In this section, we evaluate the proposed self-supervised learning-based WF attack (MRCGCN). First, a review of eight baseline methods is provided. Then, we presented the evaluation dataset, experiment settings, and performance metrics. Lastly, experiments are created to evaluate the performance of WF attacks in both closed- and open-world scenarios.

A. Baselines Overview

To perform the evaluation, we compared MRCGCN with eight state-of-the-art WF attacks. These included two supervised learning methods (Robust Fingerprinting and GraphDApp) and two unsupervised learning methods (D-PACK and IoT-Keeper). We also looked at the self-supervised learning method DCGAN, the semi-supervised learning method FS-GAN, the transfer learning method FM-CWFA, and the reinforcement learning method EDRL. We have fine-tuned them to fit this paper's task.

- 1) Robust Fingerprinting (RF) is a CNN-based traffic classifier. Predicated on observations of uplink and downlink packets in a consecutive time window, Shen et al. proposed a traffic aggregation matrix (TAM) and constructed a WF undisturbed by defense strategies [25].
- 2) GraphDApp is the first to provide a mature WF attack for recognizing DApps traffic utilizing a graph learning approach and validates its effectiveness on a real-world traffic dataset. The packet interaction behavior between clients and servers is described with a traffic interaction

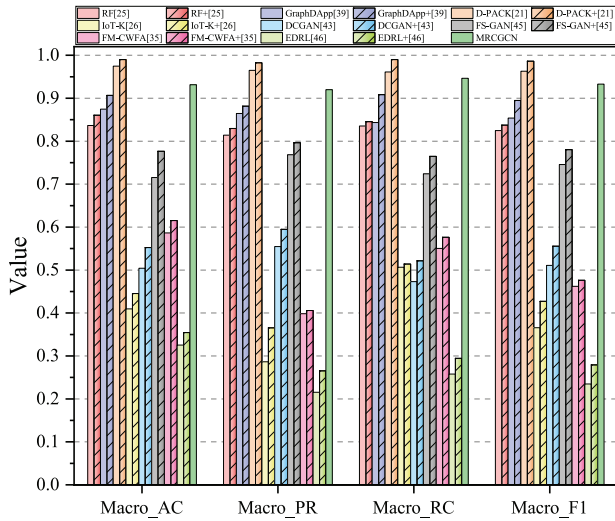


Fig. 4. Results of experiments with nine WF attacks in the closed-world. The bar charts, marked with slashes, represent the classification results of the eight compared methods optimized for application to the multi-flow task.

graph (TIG), and a highly functional GNN-based classifier is designed [39].

- 3) D-PACK proposes a fast response method for malicious traffic detection. Early detection and interception are performed by collecting a few bytes from each flow's initial few packets. It employs 1D-CNN to automatically extract features from a grayscale image and create an autoencoder-based unsupervised traffic classifier [21].
- 4) IoT-KEEPER (IoT-K) is a lightweight defense system securing the communications of IoT devices. The core detects malicious network attacks using fuzzy C-mean clustering and fuzzy interpolation. With low resources and no need for attack signatures or complex hardware, it might be implemented on various IoT devices [26].
- 5) DCGAN, a semi-supervised learning method using deep convolutional adversarial networks, solves the challenge of manually identifying large-scale traffic. The adversarial network's samples and unlabeled samples are fed into a CNN classifier for mining spatio-temporal features in packet length sequences and time series [43].
- 6) FS-GAN stacks generative adversarial networks to be compatible with structured data. It labeled samples with strong pseudo-label credibility by minimizing the Jensen-Shannon Divergence between the produced and the true sample distribution by delicate adjustments [45].
- 7) FM-CWFA, a transfer learning-based convolutional neural network, generates a WF attack using modulation to reduce classifier aging from website updates [35].
- 8) EDRL serves as a reinforcement DL strategy for WF attacks. Monte Carlo and dictionary-based learning methods expand the reward mechanism to accurately govern the multilayer perceptron neural network [46].

Many WF attacks target single-flow HTTP/HTTPS website identification or selecting the main flow (the first established or the one that transmits the most data) in the preprocessing. Only suitable in some cases, this pattern wastes significant data. We adjusted the baseline WF attack processing paradigm (Figs. 4 and 5 with plus signs) for this study. Moreover, we

TABLE III
OVERVIEW OF THE WEBSITE BROWSING TRAFFIC DATASET

Protocols	Categories	Flows	Duration	Domain name of the website
HTTP	News	103206	58 days	64365.com, people.com.cn, weather.com.cn, 39.net, duanmeiwen.com, xinhuanet.com, bendibao.com
	Shopping	846758	47 days	salvadori.tv, clothingstl.it, ideal-generator.com, laregiacars.com, steeldome.mx, petrosurltda.cl
	Software	7953	15 days	downza.cn, youdao.com
	Music	10547	25 days	kuwo.cn, kugou.com
	Searching	5482	11 days	alexa.cn
	E-Book	342973	54 days	jiwxc.net, 360doc.com, gaosan.com, zybang.com, ruiwen.com, 5068.com
	Game	249433	27 days	4399.com, yxdown.com
	Public	88525	63 days	apartamentosacachada.com, ercankocoglu.com, museojesusnazareno.es, pension-neon.com, climbingholds.ru, intrast.pl, skyhang.jp
	Blog	3564	8 days	patriciaorruent.com, blogspot.com
	Malicious	120125	39 days	0531qly.net, espad.org, 365winner.biz, 1e1v.com
HTTPS	News	268451	32 days	weibo.com, forbes.com, sina.com.cn, nih.gov, who.int, qq.com
	Shopping	754839	22 days	tmall.com, apple.com, taobao.com, jd.com, aliexpress.com
	Software	354824	18 days	apache.org, github.com, cloudflare-cn.com, sourceforge.net, stackoverflow.com
	Music	195674	29 days	spotify.com, aol.com, y.qq.com, last.fm, music.163.com
	Searching	6853	31 days	baidu.com, bing.com, google.com, yahoo.com
	Office	18468	15 days	360.cn, linkedin.com, office.com, wordpress.com
	Mailbox	8544	17 days	foxmail.com, mail.163.com, mail.qq.com, outlook.live.com
	Video	264851	26 days	zhangqi.tv, reddit.com, imdb.com, youtube.com
	Social	24693	19 days	yandex.ru, vk.com, facebook.com, instagram.com
	Payment	14750	14 days	apple.com, paypal.com, alipay.com, pay.google.com

employed stacking models to handle the data and integrated the results through voting algorithms.

B. Dataset Collection

To evaluate the efficacy of the proposed MRCGCN method, we mimic clients' website browsing behaviors in the real world and construct an HTTP and HTTPS traffic dataset comprising many typical website types. This dataset obtains ten representative types of real website traffic, such as news websites, online shopping, software services, and online music, for the plaintext HTTP/1.1 protocol and its encrypted HTTPS protocol, respectively, as detailed in Table III.

We simulated clients' browsing behavior utilizing the Chromedriver procedure and employed the tcpdump software to monitor both 80 and 443 ports under the TCP protocol. In reality, website content is dynamically updated at different frequencies, leading to significant alterations in the composition of resources inside TCP flows over different time periods. Even if only one resource on the page is changed in a single update, the pattern of the parallel flows may be completely altered under the influence of the dynamic resource allocation mechanism, hence complicating the single flow identification method. To accurately represent the comprehensive browsing behavior of a website, we record all HTTP and HTTPS traffic during the browsing session and use it as a sample, rather than concentrating solely on a singular primary flow or the flow with the highest volume of transmitted data.

The constructed dataset consists of 83 websites, most of which are typical websites in the generic service type, such as the top 100 visited online shopping websites, Tmall and Apple Mall; the online video website YouTube; and the financial payment websites Alipay and PayPal. We set different observation durations (8)-63 days) for the websites based on

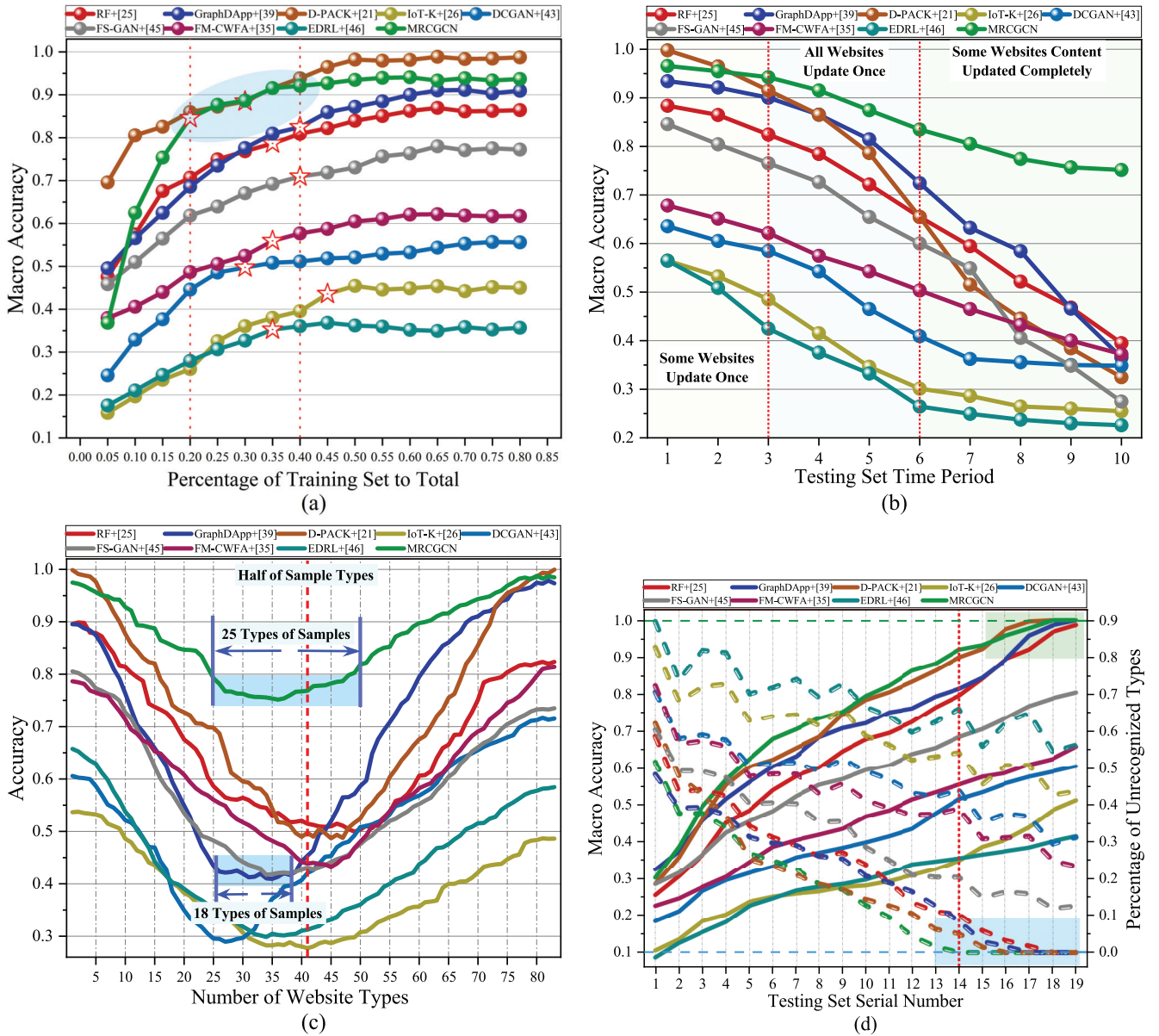


Fig. 5. Experimental results for nine WF attacks in the open-world. (a) Convergence speed test; (b) Robustness test; (c) Generalizability test; (d) Flexibility test.

their update frequency, with 500, 1,000, 2,000, or 4,000 visits. The total number of HTTP and HTTPS flows within this dataset is 3,690,513, with each website potentially connecting to 1-5 IP addresses, 3-10 domain names, 5-20 flows, and transferring 20-50 resource files during a single browsing.

C. Experiment Setup

In this subsection, we introduce the parameters involved in the workflow of the MRCGCN framework. All experiments were conducted using a GCN model built with the DGL library, running on a 12th generation Intel Core processor i9-12900K, a GeForce RTX 3090 graphics card, and 64GB of RAM.

MRCGCN is a self-supervised WF attack, where the training phase includes unsupervised graph clustering, a graph similarity algorithm for sample labeling, and supervised graph

classification. The graph clustering refines similar graph structures of related websites and provides templates for graph similarity processing to generate pseudo-labels. The clustering termination condition is based on the number of sample types. The threshold for determining graph similarity is set at 5%, meaning that the similarity between a sample and the most similar type must be at least 5% higher than other types. Otherwise, further comparison with secondary clustering cores is required.

The choice of hyperparameters is crucial for the performance of GNN. While increasing the number of layers, units per layer, and iterations can improve classification accuracy, it also increases training complexity. Hence, a balanced hyperparameter configuration is essential. Through parameter tuning on a small sample set, we selected *Adam* as the optimizer and *ReLU* as the activation function. Node feature

aggregation was chosen as the aggregation method, while node update was used as the replacement method. The optimal number of iterations was determined to be 20, the number of graph convolution layers was set to 5, with 30 hidden units per layer, and each branch contained 200 graphs.

D. Performance Metrics

In this paper, we evaluate the performance of all methods with four metrics: *Accuracy (AC)*, *Precision (PR)*, *Recall (RC)*, and *F1-score (F1)*, which are frequently utilized for 2-classification tasks. The effectiveness of nine WF attacks in multi-classification tasks is determined via the macro-average of four metrics, defined as follows:

$$\left\{ \begin{array}{l} Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \\ Precision = \frac{TP}{TP + FP} \\ Recall = \frac{TP}{TP + FN} \\ F1 - score = \frac{2 \times PR \times RC}{PR + RC} \end{array} \right. \quad (18)$$

$$\left\{ \begin{array}{l} Macro_AC = \frac{1}{n} \sum_{i=1}^n AC_i \\ Macro_PR = \frac{1}{n} \sum_{i=1}^n PR_i \\ Macro_RC = \frac{1}{n} \sum_{i=1}^n RC_i \\ Macro_F1 = \frac{1}{n} \sum_{i=1}^n F1_i \end{array} \right. \quad (19)$$

where TP , TN , FP , and FN denote the true positive, true negative, false positive, and false negative, respectively. AC_i , PR_i , RC_i , and $F1_i$ denote the *Accuracy*, *Precision*, *Recall*, and *F1-score* for class i , and $Macro_AC$, $Macro_PR$, $Macro_RC$, and $Macro_F1$ denote the arithmetic mean of four metrics for each category, respectively.

E. Closed-World Evaluation

All website traffic falls within a restricted range in the closed-world. WF attacks are tested for multi-classification accuracy in this experiment.

The multi-classification examinations evaluated nine WF attacks that classified 83 websites. We split the samples into five equal subsets. Unsupervised methods perform each subset individually and take the average classification results. We took one subset at a time without repetition as the test set and the rest as the training set in supervised learning, and the final result is the average of five classification results.

Figure 4 illustrates multi-classification results for nine WF attacks. The suggested MRCGCN, D-PACK, RF, GraphDApp, and their modifications outperform the remaining methods, with their macro-averaged classification accuracies of 0.9315, 0.9748, 0.8365, and 0.8745 for websites, respectively. The initial few website traffic packets are utilized by D-PACK to produce a grayscale image. It performs effectively with URLs, domain names, and certificates; character-level traffic elements exist strongly associated with websites. But website upgrades could readily invalidate them, rendering WF attacks unreliable.

TABLE IV
REPRESENTATION CONSTRUCTION TIME (Trc), REPRESENTATION INFERENCE TIME (Tri), AND TOTAL CLASSIFICATION TIME (Ttc) CONSUMPTION OF NINE WF ATTACKS FOR WEBSITE MULTI-CLASSIFICATION EXPERIMENTS

Methods	$Trc(s)$	$Tri(s)$	$Ttc(s)$
Robust fingerprinting+ [25]	786472	141984	928456
GraphDApp+ [39]	2015972	705950	2721922
D-PACK+ [21]	136825	85846	222671
IoT-K+ [26]	177463	73652	251115
DCGAN+ [43]	1754862	1179888	2934750
FS-GAN+ [45]	231868	45050	276918
FM-CWFA+ [35]	1408264	530934	1939198
EDRL+ [46]	893648	350429	1244077
MRCGCN	189712	869766	1059478

The starting phase of website browsing mainly transmits HTML, CSS, and JS, which are unable to describe the webpage's unique style, and traffic packet feature sequences vary only slightly. As the count and type of transmitted resources increased, website updates would deform subsequent sequences, causing GraphDApp, IoT-KEEPER, DCGAN, FS-GAN, and EDRL to lose credibility.

RF and FM-CWFA improve website coverage by matching coarse-grained direction sequences but losing recognition accuracy. The direction sequences depict an unstable webpage structure caused by inadequate website information analysis. Thus, MRCGCN fully analyzes resource types and combinations in traffic graph representation, improving stability. We then created an extensive experiment in open-world scenarios to validate. Moreover, as seen in Fig. 4, our strategy improves all baselines; hence, we would not compare the raw WF attacks in subsequent experiments.

Table IV presents the time consumption of nine WF attacks in the website multi-classification experiments. Three key time metrics are used: representation construction time (Trc), which measures the time taken from traffic capture to forming learnable representations; representation inference time (Tri), which measures the time required to classify the formed representations; and total classification time (Ttc), which is the sum of the previous two. These metrics allow for a comprehensive comparison of the time complexity across the different computational processes involved in each method.

As shown in Table IV, the methods can be categorized into four levels based on their computational complexity and time consumption. The first level includes D-PACK+, IoT-K+, and FS-GAN+, which exhibit the lowest time consumption. The second level comprises Robust Fingerprinting+, MRCGCN, and EDRL+, while the third level contains only FM-CWFA+. The final level, with the highest time consumption, includes GraphDApp+ and DCGAN+.

The first-level methods have lower computational complexity because they rely on simpler feature extraction techniques: D-PACK+ and IoT-K+ focus on character-level features, while

TABLE V

AN ABLATION EXPERIMENT ON THE CONTRIBUTION OF EACH LEVEL IN MRCGCN TO THE CLASSIFICATION PERFORMANCE

Level combinations	Macro_AC	Macro_PR	Macro_RC	Macro_F1
Resource-level	0.7354	0.6985	0.7215	0.7098
Resource and Flow-levels	0.8683	0.8421	0.8598	0.8509
Resource and Host-levels	0.7934	0.7798	0.8124	0.7958
MRCGCN	0.9315	0.9199	0.9465	0.9395

FS-GAN+ is based on statistical features like packet length. In comparison, MRCGCN is on par with methods such as EDRL+, which uses packet length sequences, and Robust Fingerprinting+, which relies on packet direction sequences. These methods capture more sophisticated spatio-temporal features of website traffic, requiring more computational resources than first-level methods but significantly less than those in the higher levels.

MRCGCN's representation construction time (Trc) is comparable to the first-level methods and even shorter than that of FS-GAN+, thanks to the multi-flow processing approach, which treats all flows during browsing as a single sample and represents them as a graph. Additionally, the developed graph partitioning algorithm reduces Trc by utilizing multi-threaded processing. While MRCGCN exhibits greater complexity in graph construction and point-by-point comparison than Graph-DApp+, another graph-based method, their representation inference time (Tri) is similar, demonstrating that our hierarchical pooling algorithm is both robust and efficient.

To evaluate the contribution of each level in the MRCGCN on the model's classification performance, we designed an ablation experiment, wherein we evaluated the use of solely resource-level features, the combination of resource-level and flow-level features, the combination of resource-level and host-level features, and the utilization of all level features.

Table V presents the classification results of MRCGCN for websites utilizing various combinations of traffic features from different levels. Overall, all three levels contribute positively to the model's classification of websites; thus, the ablation of any level results in diminished overall performance. The results of the four experiments indicate that the resource level contributes the most. Using only resource-level features, the model's classification accuracy has reached 0.7354, which is 0.1961 lower than that of the complete MRCGCN. The classification accuracies of the combination of resource-level and flow-level features and the combination of resource-level and host-level features have improved to 0.1329 and 0.058 over that of using only the resource-level, with an improvement ratio of 18.07% and 7.89%, respectively. This is consistent with our perception that flow-level presents a more fine-grained spatio-temporal feature of traffic than host-level.

F. Open-World Evaluation

Realistic website environments, where the count and type of websites are not predetermined and their content is continuously updated, would age the classifiers. We designed four open-world experiments to investigate the convergence speed, robustness, generalizability, and flexibility of WF attacks.

1) *Convergence Speed Test*: Training set scale has a tremendous impact on the ML's performance. More isostructural samples raise the probability of mining core features and reaching the model's stable peak faster.

We first selected 80% of the samples from each website to form a temporary training set, with the remaining 20% used as a test set. In the first round, we trained the supervised methods using 5% of the temporary training set and evaluated them on the test set. In each subsequent round, we added an additional 5% of samples to the training set, incrementally increasing the training data until reaching 80%. Each experiment was evaluated using ten-fold cross-validation. Unsupervised methods used an equivalent test set for evaluation.

The convergence speed test results for the WF attacks are shown in Fig. 5(a). The suggested MRCGCN has the fastest convergence speed, requiring only three data additions to reach dynamic equilibrium (0.9 times the highest value, the star-shaped point on the curve in the figure). Its macro-averaged classification accuracy increased from 36.8% at 5% data to 84.6% at 20%. Next, D-PACK+ and DCGAN+ require 30% data to reach 0.9. Although the maximum classification accuracy of 98.9% for D-PACK+ is slightly higher than that of 94.1% for MRCGCN, their classification accuracies are essentially equal when the data size is 0.2 to 0.4 of the entire. In large-scale scenarios, the methods that achieve dynamic equilibrium quicker require fewer computations and are more adaptable to new changes.

2) *Robustness Test*: Website updates can induce concept drift, deteriorating the efficacy of website browsing traffic classifiers. To assess the robustness of the nine WF attacks against concept drift, we partitioned the dataset into ten subsets based on the capture dates of the traffic data.

For supervised learning methods, 80% of each website's first-time period samples were utilized as the training set and the rest as the test set.

For unsupervised learning methods, we utilized type IDs to determine cluster similarity. Then the remaining samples are assessed using similarity as a criterion. Notably, there is an independent similarity for each category in the clustering procedure. Global similarity, the smallest of these, was selected and used as a constraint for subsequent experiments.

In Fig. 5(b), the WF attacks' robustness tests are shown. The macro-averaged accuracy of all methods over time periods illustrates that website updates affect classifier performance. Several WF attacks' categorization accuracy dropped below 0.5 even in the final, making them untrustworthy. MRCGCN is the most stable of all methods. Even when some website contents changed, it could over 0.9 accuracy (yellow region in the figure). Among all webpages updated once, it has the slowest classifier aging. While most of the website's resources changed (images were replaced, but the format remained unchanged), it maintained a classification accuracy of 0.75, with some websites being captured over 50 days. In other experiments, D-PACK+, which slightly outperforms MRCGCN, loses classification accuracy due to changes in crucial data like URLs. The results confirm MRCGCN's reliability, which utilizes resource attributes and webpage layouts to construct classifiers that effectively resist concept drift.

3) *Generalizability Test*: Accurately identifying unlearned websites poses significant challenges for WF attacks, limiting their applicability in automated and large-scale scenarios. In this experiment, we first selected 80% of the samples from each website type to form an alternative training dataset and used the remaining 20% as a test set. In the first round, we trained the model using samples from any one website type. During testing, the generated traffic classifier was used to recognize all types of mixed website traffic. In subsequent rounds, we added one additional website type to the training set, randomly selected without replacement, until all types were included. Samples whose similarity did not meet the minimum categorization threshold were classified as unknown. For unsupervised learning methods, we determined cluster similarity using type IDs. We assessed the remaining samples based on their similarity to these clusters. Notably, in the clustering procedure, each category has an independent similarity measure. The global similarity threshold, defined as the smallest of these individual similarities, was selected and used as a constraint for subsequent experiments.

Unsupervised learning algorithms without supervised signals struggle with generalizability tests. During feature matching, all test set samples are compared to clustering centers. When similarity exceeds independent similarity, the category is identified. When several similar categories exist, the sample is placed in the highest independent similarity class.

This subtest first classifies samples using multi-classification. Subsequently, it could be regarded as a 2-classification, where all learned types from the training would be merged and analyzed uniformly as positives. The results are also reported using 2-classification metrics.

Figure 5(c) shows the WF attacks' generalizability tests. It demonstrates that WF attacks hardly classify unlearned or unlabeled samples but affect classifiers differently. As seen in the figure, WF attacks' categorization accuracy changes in three stages. The suggested MRCGCN degrades classification accuracy at the slowest rate as sample types grow. Around one-third of the time, it plateaus and stops falling. The classifier enters the second stage when the ratio of new types to categories learned is no longer lopsided. In the third stage, the classifier learns the majority of types, enabling it to classify the test set successfully. Eventually, it approximates the closed-world scenario, and classifier performance peaks. MRCGCN surpasses the other WF attacks in classification accuracy and dynamic stability period, whereas the others fall quickly.

4) *Flexibility Test*: In the real world, the types of websites and their proportions are erratic and difficult to predict. For website traffic identification in large-scale scenarios, the WF attack should automatically identify categories and extract interpretable website traffic features. To evaluate this capability, we divided the dataset into twenty equal parts without distinguishing website categories. In the first round, we used two parts for training and testing, respectively. In subsequent rounds, we iteratively updated the model with unrecognized samples by adding one more part for testing, repeating this process until all samples were traversed.

Since supervised learning can't deduce unlabeled inputs, we trained the models with labeled unsupervised learning examples. The first stage extracts two groups for training and testing. In the second step, the model was updated with unidentifiable samples. A non-putback random group sample is taken for testing. Repeat the second step until all samples are traversed. Calculating classification results skips updating the model for the set while all samples are recognized.

Figure 5(d) shows the WF attacks' flexibility tests. The solid lines show the macro-averaged accuracy in each test set. The dashed line reflects the unrecognized type proportion of the test set's total types. The figure reveals that MRCGCN, D-PACK+, RF+, and GraphDApp+ perform better. After 0.3 initial recognition accuracies, all improve steadily, with new samples reaching complete classification in the final two subtests. This proves they could extract effective website traffic features, and MRCGCN recognizes new samples best. It categorized all samples with the 14th test, making it the first one to finish classification. Large-scale and automated scenarios require this capability, improving the traffic inspection system's probability of finding unexpected services.

VI. CONCLUSION AND DISCUSSION

In this study, we propose a novel self-supervised learning-based WF attack called the multi-level resource-coherented graph convolutional neural network (MRCGCN). The graph representation is built on the notion that website resource types and layouts rarely change. MRCGCN could precisely mine a website's inherent properties and generate an accurate and stable website traffic classifier utilizing the kernel function for echelon-ordered graphs, the coalescent hierarchical clustering algorithm, the affiliation-driven hierarchical graph-based pooling method, and the directed aggregation function for node features based on the hierarchy deviation. In real-world datasets, MRCGCN outperforms state-of-the-art WF attacks with strong convergence speed, robustness, generalizability, and flexibility.

MRCGCN effectively classifies HTTP and HTTPS website traffic for HTTP/1.1 websites. However, the increasing adoption of HTTP/2.0 and HTTP/3.0 introduces multiplexing mechanisms that interleave packets from different resources, complicating the traffic representation sequences. This multiplexing renders MRCGCN less effective in accurately identifying website resources.

To enhance the applicability of the proposed method, we briefly discuss potential improvements to the model. In HTTP/2.0 and HTTP/3.0, content encryption and multiplexing limit our ability to precisely identify the start and end positions of each resource transmission. As a result, a single packet may belong to multiple resources simultaneously. In the graph representation, this means a packet-level node might be connected to several resource-level nodes concurrently. To address this challenge, we can implement a hypergraph model to reconstruct the representations of HTTP/2.0 and HTTP/3.0 website browsing traffic and extract key characteristics using suitable hypergraph algorithms.

Despite the lack of clear page layouts and the challenges in maintaining consistent and singular mappings, certain

distinctive characteristics remain. These include the transmission sequence of resource zones, the number of concurrently transmitted resources, and the points at which each resource concludes its transmission. While these features are more complex, they can still effectively differentiate between various website types. We believe that integrating knowledge of protocol operational mechanisms, real traffic patterns, and advanced graph algorithms will enable graph learning-based WF attacks to identify traffic from websites using newer protocols.

REFERENCES

- [1] T. Berners-Lee, R. T. Fielding, and H. F. Nielsen, *Hypertext Transfer Protocol-HTTP/1.1*, document RFC 2068, 1997, pp. 1–162.
- [2] D. Wagner and B. Schneier, “Analysis of the SSL 3.0 protocol,” in *Proc. 2nd Conf. 2nd USENIX Workshop Electron. Commerce*, vol. 2, Nov. 1996, p. 4.
- [3] M. Shen et al., “Machine learning-powered encrypted network traffic analysis: A comprehensive survey,” *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023, doi: 10.1109/COMST.2022.3208196.
- [4] N. P. Hoang, A. A. Niaki, P. Gill, and M. Polychronakis, “Domain name encryption is not enough: Privacy leakage via IP-based website fingerprinting,” 2021, *arXiv:2102.08332*.
- [5] M. Zouina and B. Outtaj, “A novel lightweight URL phishing detection system using SVM and similarity index,” *Hum.-Centric Comput. Inf. Sci.*, vol. 7, no. 1, pp. 1–13, Dec. 2017, Art. no. 17, doi: 10.1186/s13673-017-0098-1.
- [6] H. Yamauchi, A. Nakao, M. Oguchi, S. Yamamoto, and S. Yamaguchi, “A study on service identification based on server name indication analysis,” in *Proc. 7th Int. Symp. Comput. Netw. Workshops (CANDARW)*, Nov. 2019, pp. 470–474, doi: 10.1109/CANDARW.2019.00089.
- [7] R. Lua and K. C. Yow, “Mitigating DDoS attacks with transparent and intelligent fast-flux swarm network,” *IEEE Netw.*, vol. 25, no. 4, pp. 28–33, Aug. 2011, doi: 10.1109/MNET.2011.5958005.
- [8] E. Hahne, “Round-robin scheduling for max-min fairness in data networks,” *IEEE J. Sel. Areas Commun.*, vol. 9, no. 7, pp. 1024–1039, Sep. 1991, doi: 10.1109/49.103550.
- [9] A. Saverimoutou, B. Mathieu, and S. Vaton, “Influence of internet protocols and CDN on web browsing,” in *Proc. 10th IFIP Int. Conf. New Technol., Mobility Security (NTMS)*, 2019, pp. 1–5, doi: 10.1109/NTMS.2019.8763827.
- [10] T.-D. Pham, T.-L. Ho, T. Truong-Huu, T.-D. Cao, and H.-L. Truong, “MAppGraph: Mobile-app classification on encrypted network traffic using deep graph convolution neural networks,” in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2021, pp. 1025–1038, doi: 10.1145/3485832.3485925.
- [11] A. W. Moore and K. Papagiannaki, “Toward the accurate identification of network applications,” in *Proc. Int. Workshop Passive Active Netw. Meas.* Berlin, Germany: Springer-Verlag, 2005, pp. 41–54, doi: 10.1007/978-3-540-31966-5_4.
- [12] T. T. T. Nguyen and G. Armitage, “A survey of techniques for internet traffic classification using machine learning,” *IEEE Commun. Surveys Tuts.*, vol. 10, no. 4, pp. 56–76, 4th Quart., 2008, doi: 10.1109/SURV.2008.080406.
- [13] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, “AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic,” in *Proc. IEEE Eur. Symp. Secur. Privacy*, Mar. 2016, pp. 439–454, doi: 10.1109/EuroSP.2016.40.
- [14] M. Shen, Y. Liu, L. Zhu, X. Du, and J. Hu, “Fine-grained webpage fingerprinting using only packet length information of encrypted traffic,” *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2046–2059, 2021, doi: 10.1109/TIFS.2020.3046876.
- [15] A. Panchenko et al., “Website fingerprinting at internet scale,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, 2016, pp. 1–15, doi: 10.14722/ndss.2016.23477.
- [16] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, “Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail,” in *Proc. IEEE Symp. Secur. Privacy*. San Francisco, CA, USA: IEEE Computer Society, May 2012, pp. 332–346, doi: 10.1109/SP.2012.28.
- [17] P. Sirinam, M. Imani, M. Juarez, and M. Wright, “Deep fingerprinting: Undermining website fingerprinting defenses with deep learning,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Toronto, ON, Canada, Oct. 2018, pp. 1928–1943, doi: 10.1145/3243734.3243768.
- [18] S. Rezaei, B. Kroencke, and X. Liu, “Large-scale mobile app identification using deep learning,” *IEEE Access*, vol. 8, pp. 348–362, 2020, doi: 10.1109/ACCESS.2019.2962018.
- [19] A. L. Varna and M. Wu, “Modeling and analysis of correlated binary fingerprints for content identification,” *IEEE Trans. Inf. Forensics Security*, vol. 6, pp. 1146–1159, 2011, doi: 10.1109/TIFS.2011.2152394.
- [20] Y. Qin, W. Wang, S. Zhang, and K. Chen, “An exploit kits detection approach based on http message graph,” *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3387–3400, 2021, doi: 10.1109/TIFS.2021.3080082.
- [21] R.-H. Hwang, M.-C. Peng, C.-W. Huang, P.-C. Lin, and V.-L. Nguyen, “An unsupervised deep learning model for early network traffic anomaly detection,” *IEEE Access*, vol. 8, pp. 30387–30399, 2020, doi: 10.1109/ACCESS.2020.2973023.
- [22] P. Sirinam, N. Mathews, M. S. Rahman, and M. Wright, “Triplet fingerprinting: More practical and portable website fingerprinting with N-shot learning,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1131–1148, doi: 10.1145/3319535.3354217.
- [23] Y. Wang, H. Xu, Z. Guo, Z. Qin, and K. Ren, “SnWF: Website fingerprinting attack by ensembling the snapshot of deep learning,” *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1214–1226, 2022, doi: 10.1109/TIFS.2022.3158086.
- [24] M. Jiang et al., “Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks,” *Comput. Netw.*, vol. 217, Nov. 2022, Art. no. 109309, doi: 10.1016/j.comnet.2022.109309.
- [25] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, “Subverting website fingerprinting defenses with robust traffic representation,” in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 607–624. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/shen-meng>
- [26] I. Hafeez, M. Antikainen, A. Y. Ding, and S. Tarkoma, “IoT-KEEPER: Detecting malicious IoT network activity using online traffic analysis at the edge,” *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 1, pp. 45–59, Mar. 2020, doi: 10.1109/TNSM.2020.2966951.
- [27] X. Han, T. Lv, L. Wei, Y. Wu, J. Liu, and C. Wang, “A model for website anomaly detection based on log analysis,” in *Proc. IEEE 3rd Int. Conf. Cloud Comput. Intell. Syst.*, Nov. 2014, pp. 604–608, doi: 10.1109/CCIS.2014.7175806.
- [28] X. Yan, Y. Xu, B. Cui, S. Zhang, T. Guo, and C. Li, “Learning URL embedding for malicious website detection,” *IEEE Trans. Ind. Informat.*, vol. 16, no. 10, pp. 6673–6681, Oct. 2020, doi: 10.1109/TII.2020.2977886.
- [29] T. Kiziloren and E. Germen, “Network traffic classification with self organizing maps,” in *Proc. 22nd Int. Symp. Comput. Inf. Sci.*, Nov. 2007, pp. 1–5, doi: 10.1109/fiscis.2007.4456852.
- [30] C. S. Sastry, S. Rawat, A. K. Pujari, and V. P. Gulati, “Network traffic analysis using singular value decomposition and multiscale transforms,” *Inf. Sci.*, vol. 177, no. 23, pp. 5275–5291, Dec. 2007, doi: 10.1016/j.ins.2006.07.007.
- [31] C. Yao, Y. Yang, K. Yin, and J. Yang, “Traffic anomaly detection in wireless sensor networks based on principal component analysis and deep convolution neural network,” *IEEE Access*, vol. 10, pp. 103136–103149, 2022, doi: 10.1109/ACCESS.2022.3210189.
- [32] H. Alqahtani et al., “Evolutionary algorithm with deep auto encoder network based website phishing detection and classification,” *Appl. Sci.*, vol. 12, no. 15, p. 7441, Jul. 2022, doi: 10.3390/app12157441.
- [33] S. Zhang, J. Yin, Z. Li, R. Yang, M. Du, and R. Li, “Node-imbalance learning on heterogeneous graph for pirated video website detection,” in *Proc. IEEE 25th Int. Conf. Comput. Support. Cooperat. Work Design (CSCWD)*, Sep. 2022, pp. 834–840, doi: 10.1109/CSCWD54268.2022.9776224.
- [34] F. Nie, W. Liu, G. Liu, and B. Gao, “M2 VT-IDS: A multi-task multi-view learning architecture for designing IoT intrusion detection system,” *Internet Things*, vol. 25, Apr. 2024, Art. no. 101102, doi: 10.1016/j.iot.2024.101102.
- [35] T. Pan, Z. Tang, and D. Xu, “A practical website fingerprinting attack via CNN-based transfer learning,” *Mathematics*, vol. 11, no. 19, p. 4078, Sep. 2023, doi: 10.3390/math11194078.
- [36] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, “FS-Net: A flow sequence network for encrypted traffic classification,” in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Jun. 2019, pp. 1171–1179, doi: 10.1109/INFOCOM.2019.8737507.

- [37] V. Rimmer, D. Preuveneers, M. Juarez, T. V. Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15, doi: 10.14722/ndss.2018.23105.
- [38] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8, doi: 10.23919/ITUK50268.2020.9303204.
- [39] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021, doi: 10.1109/TIFS.2021.3050608.
- [40] B. Gao, W. Liu, G. Liu, and F. Nie, "Resource knowledge-driven heterogeneous graph learning for website fingerprinting," *IEEE Trans. Cognit. Commun. Netw.*, vol. 10, no. 3, pp. 968–981, Jun. 2024, doi: 10.1109/TCCN.2024.3350531.
- [41] A. Hintz, "Fingerprinting websites using traffic analysis," in *Proc. Int. Symp. Privacy Enhancing Technol.*, Jan. 2003, pp. 171–178.
- [42] P. Perera, Y.-C. Tian, C. Fidge, and W. Kelly, "A comparison of supervised machine learning algorithms for classification of communications network traffic," in *Proc. Int. Conf. Neural Inf. Process.* Cham, Switzerland: Springer, 2017, pp. 445–454.
- [43] A. S. Iliyasa and H. Deng, "Semi-supervised encrypted traffic classification with deep convolutional generative adversarial networks," *IEEE Access*, vol. 8, pp. 118–126, 2020, doi: 10.1109/ACCESS.2019.2962106.
- [44] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18, doi: 10.14722/ndss.2020.24412.
- [45] Y. Xiao et al., "Distributed traffic synthesis and classification in edge networks: A federated self-supervised learning approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1815–1829, Feb. 2023, doi: 10.1109/TMC.2023.3240821.
- [46] N. M. Balamurugan, M. Adimoolam, M. H. Alsharif, and P. Uthasakul, "A novel method for improved network traffic prediction using enhanced deep reinforcement learning algorithm," *Sensors*, vol. 22, no. 13, p. 5006, Jul. 2022, doi: 10.3390/s22135006.
- [47] X. Wang, S. Chen, and J. Su, "App-net: A hybrid neural network for encrypted mobile traffic classification," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Jun. 2020, pp. 424–429, doi: 10.1109/INFOCOMWKSHPS50562.2020.9162891.
- [48] N. Mathews, J. K. Holland, S. E. Oh, M. S. Rahman, N. Hopper, and M. Wright, "SoK: A critical evaluation of efficient website fingerprinting defenses," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 969–986, doi: 10.1109/SP46215.2023.10179289.
- [49] V. L. Pochat, T. V. Goethem, and W. Joosen, "Evaluating the long-term effects of parameters on the characteristics of the tranco top sites ranking," in *Proc. 12th USENIX Workshop Cyber Secur. Experimentation Test (CSET)*, Jan. 2019, pp. 1–8. [Online]. Available: <https://www.usenix.org/conference/cset19/presentation/lepochat>
- [50] V. L. Pochat, T. V. Goethem, S. Tajalizadehkhoob, M. Korczynski, and W. Joosen. (Aug. 10, 2021). *Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation*. [Online]. Available: <https://github.com/DistriNet/tranco-list>
- [51] X. Tan et al., "Inter-flow spatio-temporal correlation analysis based website fingerprinting using graph neural network," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7619–7632, 2024, doi: 10.1109/TIFS.2024.3441935.
- [52] J. Lu et al., "GAP-WF: Graph attention pooling network for fine-grained SSL/TLS website fingerprinting," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 1–8, doi: 10.1109/IJCNN52387.2021.9533543.
- [53] M. Li et al., "Guest editorial: Deep neural networks for graphs: Theory, models, algorithms, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 4367–4372, Apr. 2024, doi: 10.1109/TNNLS.2024.3371592.
- [54] J. Li, R. Zheng, H. Feng, M. Li, and X. Zhuang, "Permutation equivariant graph framelets for heterophilous graph learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11634–11648, Sep. 2024, doi: 10.1109/TNNLS.2024.3370918.
- [55] M. Li, L. Zhang, L. Cui, L. Bai, Z. Li, and X. Wu, "BLoG: Bootstrapped graph representation learning with local and global regularization for recommendation," *Pattern Recognit.*, vol. 144, Dec. 2023, Art. no. 109874, doi: 10.1016/j.patcog.2023.109874.
- [56] K. Huang, Y. G. Wang, M. Li, and A. P. Liò, "How universal polynomial bases enhance spectral graph neural networks: Heterophily, over-smoothing, and over-squashing," 2024, *arXiv:2405.12474*.
- [57] C. Huang, M. Li, F. Cao, H. Fujita, Z. Li, and X. Wu, "Are graph convolutional networks with random weights feasible?" *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 2751–2768, Mar. 2023, doi: 10.1109/TPAMI.2022.3183143.



Bo Gao (Graduate Student Member, IEEE) received the B.S. degree in physics from Jilin University, Jilin, in 2016, and the M.Eng. degree in control engineering from Nanjing University of Science and Technology, Nanjing, China, in 2020, where he is currently pursuing the Ph.D. degree. His research interests include networks and information security, graph neural networks, and encrypted traffic classification.



Weiwei Liu (Member, IEEE) received the B.S. degree in automation and the Ph.D. degree in control science and engineering from Nanjing University of Science and Technology, China, in 2010 and 2015, respectively. From 2014 to 2015, he was a Visiting Scholar with the Department of Computer Science, University of California at Davis. He is currently an Associate Professor with Nanjing University of Science and Technology. His research interests include signal processing and network traffic analysis.



Guangjie Liu (Member, IEEE) received the B.S. degree in electrical and computer engineering and the Ph.D. degree in control science and engineering from Nanjing University of Science and Technology, China, in 2002 and 2007, respectively. From 2016 to 2017, he was a Visiting Scholar with the Department of Computer Science, University of California at Davis. He is currently a Professor with Nanjing University of Information Science and Technology. His research interests include networks and information security.



Fengyuan Nie (Graduate Student Member, IEEE) received the B.S. degree in automation from Shanghai University of Electric Power, China, in July 2020. He is currently pursuing the Ph.D. degree with Nanjing University of Science and Technology. His research interests include the Internet of Things, network traffic classification, and deep learning.



Jianan Huang (Graduate Student Member, IEEE) received the B.S. degree in electronic and information engineering from Zhejiang Agriculture and Forestry University in 2021. He is currently pursuing the Ph.D. degree with Nanjing University of Science and Technology. His research interests include network traffic analysis and network covert channels.